

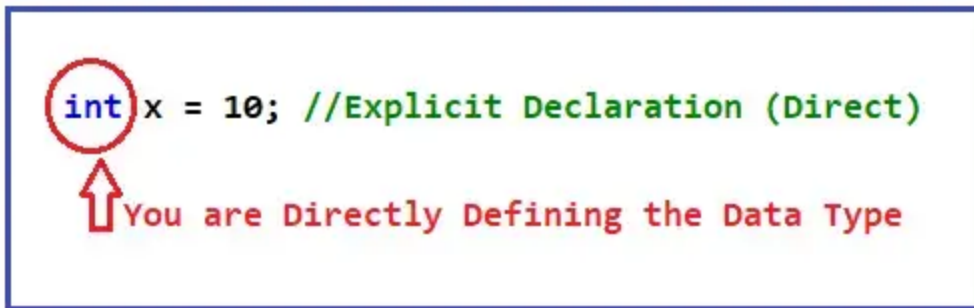
Var Keyword in C# with Examples

In this article, I am going to discuss **Var Keyword in C#** with Examples. Please read our previous article where we discussed [Dynamic Type in C#](#) with Examples. At the end of this article, you will understand the need and use of the var keyword in C#.

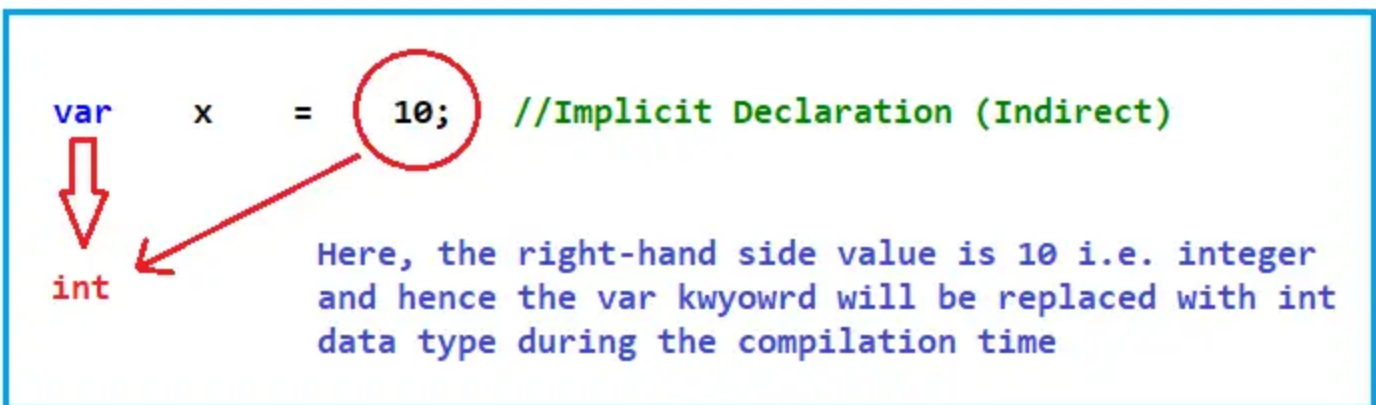
Var Keyword in C#:

In C# 3.0, the var keyword has been introduced to declare the **Implicitly Typed Local Variables** without specifying an explicit type. The type of local variables will automatically determine by the compiler based on the right-hand side assigned value of the initialization statement. Let us understand the var keyword with some examples.

When we declare a variable by using a simple data type like `int x = 10;` then it is called an **Explicit Declaration** of data type. In other words, we can say that it is a **Direct Declaration** of data type. Here, we are actually specifying the data type we are interested in i.e. int. And this is the most common way of defining a data type. For a better understanding, please have a look at the below example.



Now, let us try to understand, what happens when we start using the var keyword. So, when we define a data type using the var keyword such as `var x = 10;` then we are actually defining the data type **Indirectly or Implicitly**. In other words, when we use the var keyword, the compiler looks at the data which is present on the right-hand side and creates the appropriate data type based on the assigned value during the compilation time. For a better understanding, please have a look at the below image. In this case, the **value 10** represents the integer number and hence the var keyword replaces with int during the compilation time.



Putting it in simple words, the var keyword is not something like an object which can point to any other data during run time. Once the data type is confirmed by looking at the right-hand side data, it will only point to the valid data as per the data type. For example, in this case, the var x will always point to numeric integer values only. So, now let us define the final definition for the var keyword in C#.

What is Var in C#?

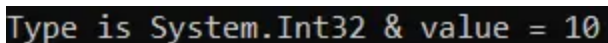
Var keyword is an implicit way or you can say an indirect way of defining data type. In simple words, when we use the var keyword to define the data type, then by looking at the data on the right-hand side of the assignment statement, the left-hand-side data type will be defined by the compiler during the generation of IL (Intermediate Language) code i.e. at the time of compilation.

Example to understand var keyword in C#:

Var keyword defines the data type statically i.e. not on run time. Let us prove this. Please have a look at the below code. Here, we are simply defining one variable using the var keyword and assigning the value 10. Then we are printing the type of the data type using the GetType method. As the value 10 is of integer type, while generating the IL code, the compiler will convert the var keyword to int data type.

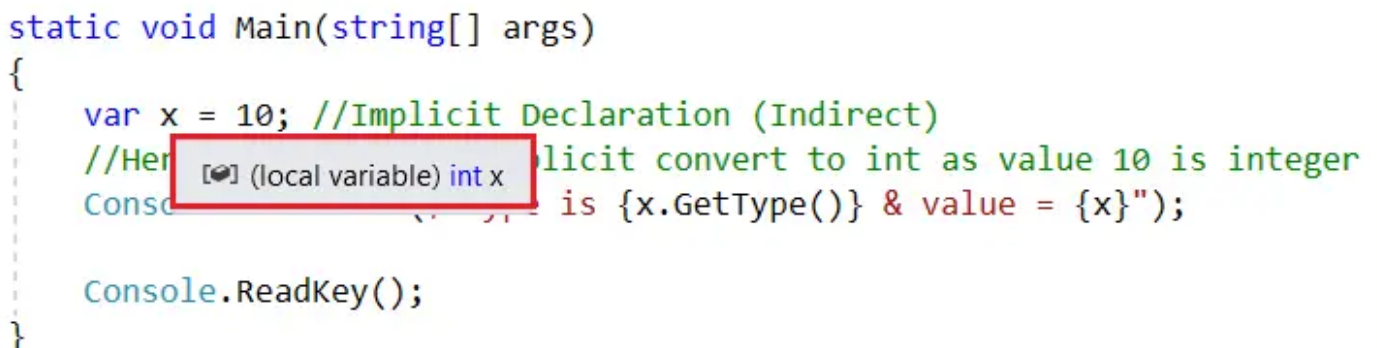
```
using System;
namespace VarKeywordDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            var x = 10; //Implicit Declaration (Indirect)
            //Here var data type implicit convert to int as value 10 is integer
            Console.WriteLine($"Type is {x.GetType()} & value = {x}");
            Console.ReadKey();
        }
    }
}
```

Now, run the above code and you will see that it will print the type as Int as shown in the below image.



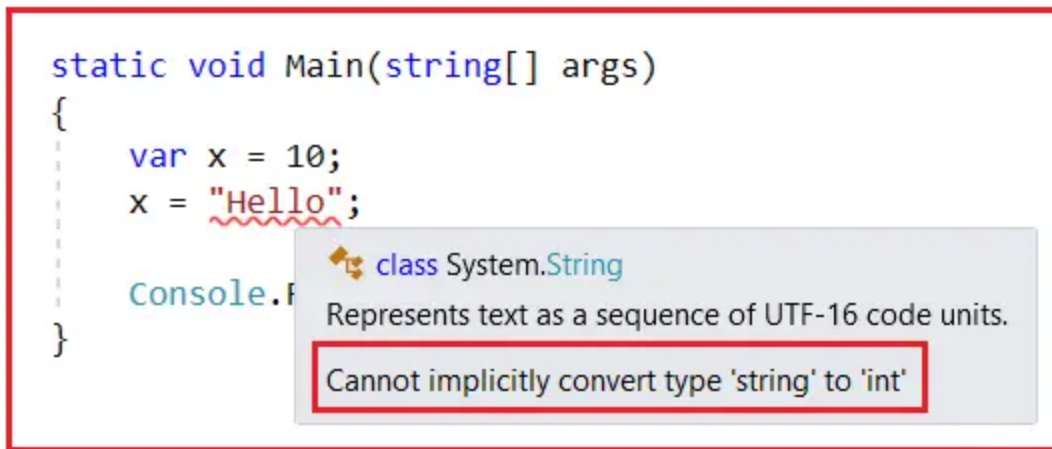
Type is System.Int32 & value = 10

This conversion happened at the compilation time. If you move the mouse pointer over the variable x, then it will show you that the x is a local variable of type int as shown in the below image.



```
static void Main(string[] args)
{
    var x = 10; //Implicit Declaration (Indirect)
    //Here var data type implicit convert to int as value 10 is integer
    Console.WriteLine($"Type is {x.GetType()} & value = {x}");
    Console.ReadKey();
}
```

As here the data type is **int** and that is decided during the compilation time, so you cannot store other types of values in it. For example, if you try to store a string value in the x variable then you will get a compile-time error as shown in the below code.



As you can see, we are getting a compilation error as **Cannot implicitly convert the type 'string' to 'int'**. This is because the data type of x is decided as int during the compilation time and hence we cannot store string value into it.

Example to Prove Var Defined the Data Type at Compile Time:

So, the var keyword statically defined the data type i.e. at compilation time, not at runtime. Let us prove this. Please modify the code as follows and then build the solution.

```
using System;
namespace VarKeywordDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            var x = 10;
            Console.ReadKey();
        }
    }
}
```

Now, build the solution. And once you build the Project an assembly (with extension EXE) will be generated inside the Project's **bin=> Debug** location as shown in the below image.

This PC > DATA (D:) > Projects > Cpp > VarKeywordDemo > VarKeywordDemo > bin > Debug

Name	Date modified	Type	Size
VarKeywordDemo	12-07-2022 12:43	Application	5 KB
VarKeywordDemo.exe.config	12-07-2022 11:28	XML Configuration...	1 KB
VarKeywordDemo.pdb	12-07-2022 12:43	Program Debug D...	16 KB

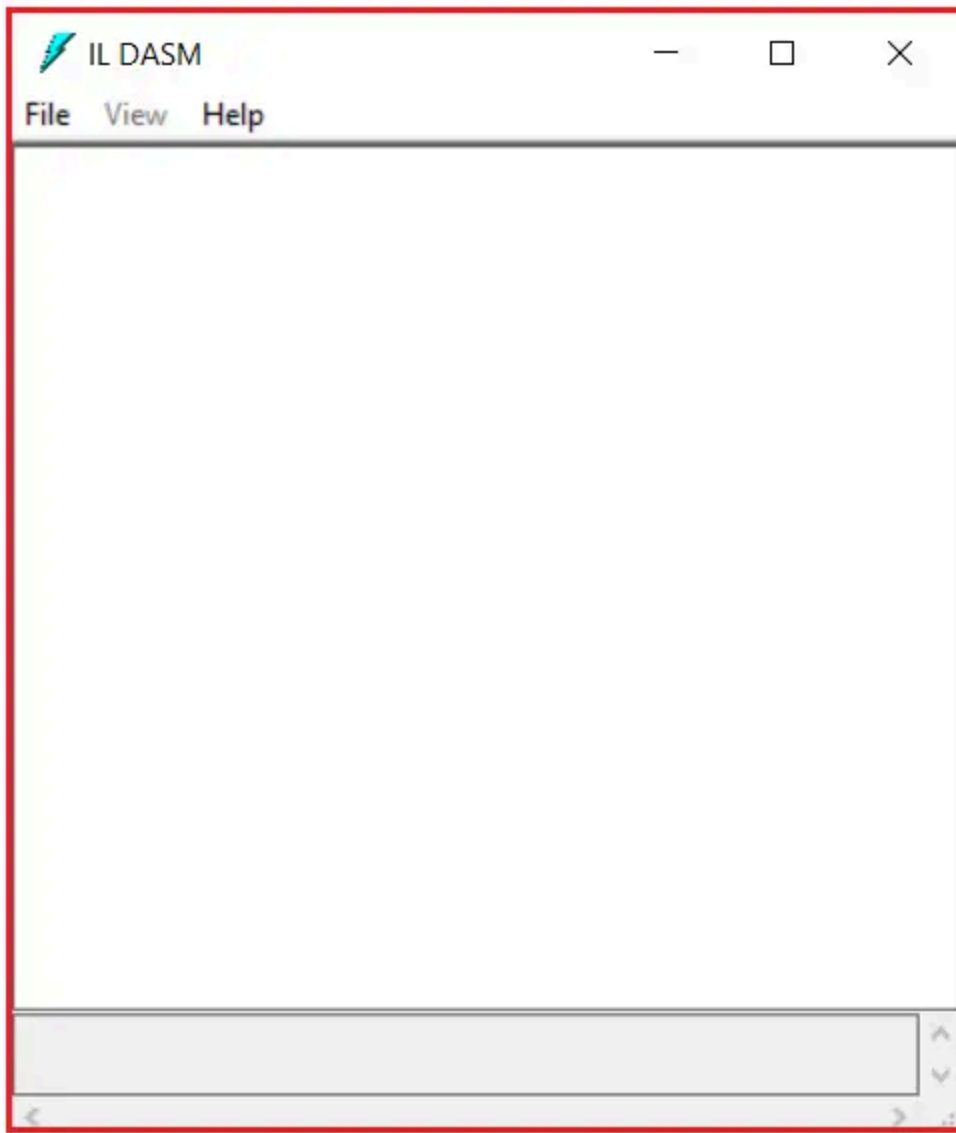
So, basically, in my machine, in the following location, the **VarKeywordDemo.exe** assembly is created. Copy the location.

D:\Projects\Cpp\VarKeywordDemo\VarKeywordDemo\bin\Debug

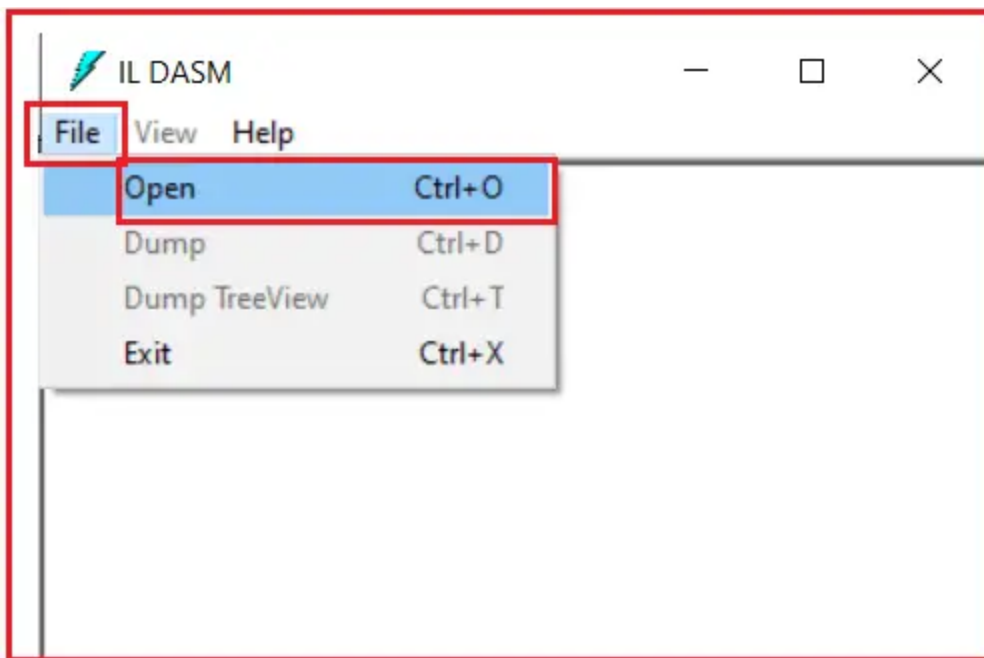
Now, open the Visual Studio Command Prompt in Administrator mode and then type ILDASM and press the enter button as shown in the below image.

```
Developer Command Prompt for VS 2017
*****
** Visual Studio 2017 Developer Command Prompt v15.9.36
** Copyright (c) 2017 Microsoft Corporation
*****
C:\Program Files (x86)\Microsoft Visual Studio\2017\Community>ildasm
```

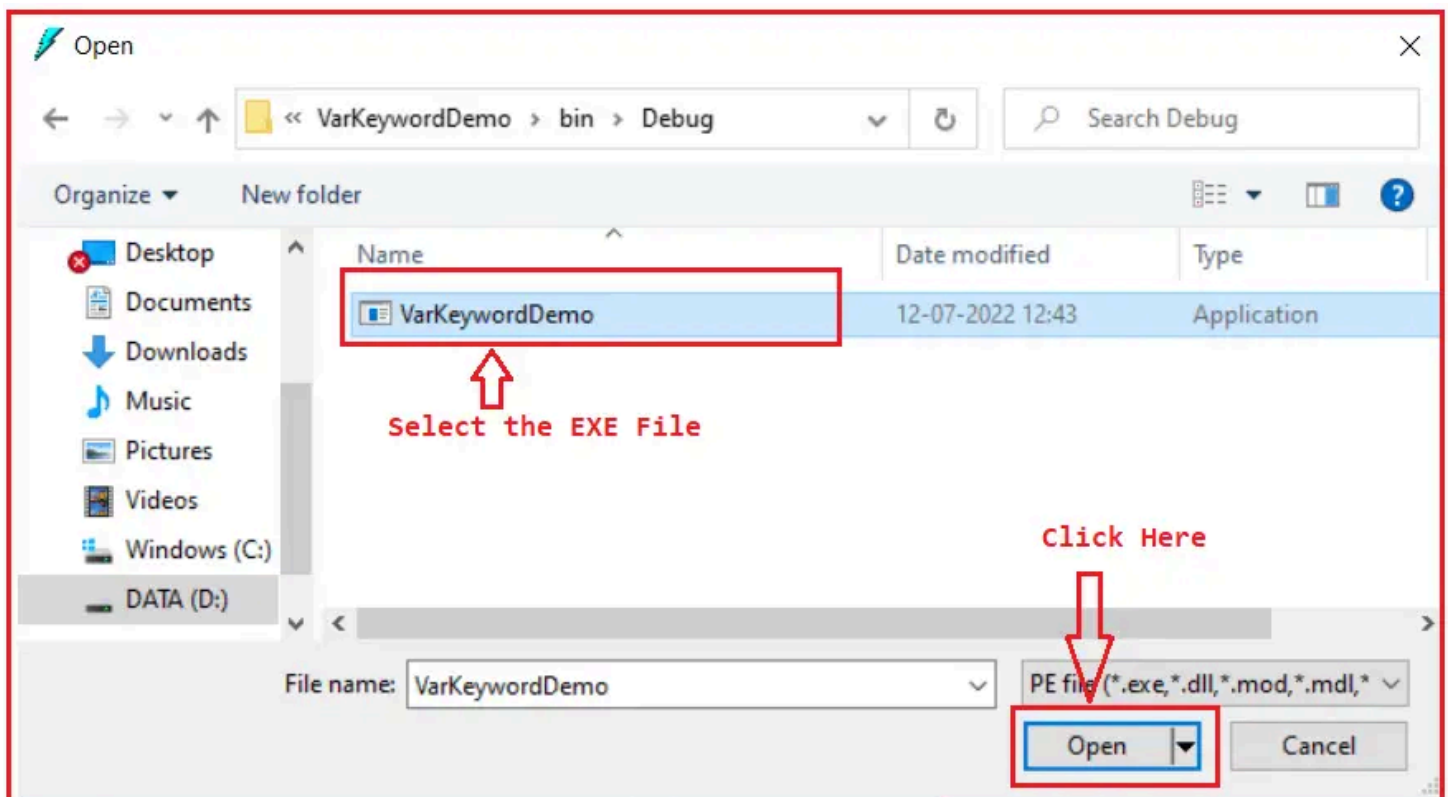
Once you press the enter button, it will open the ILDASM window as shown in the below image.



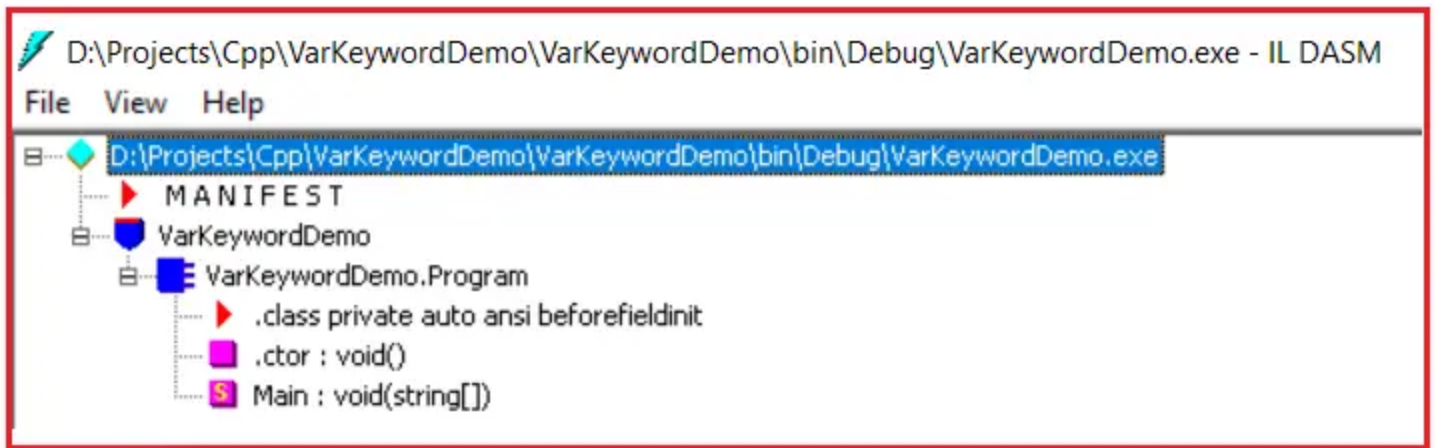
Now, open the EXE file using ILDASM. To do so, select **File => Open** from the context menu as shown in the below image.



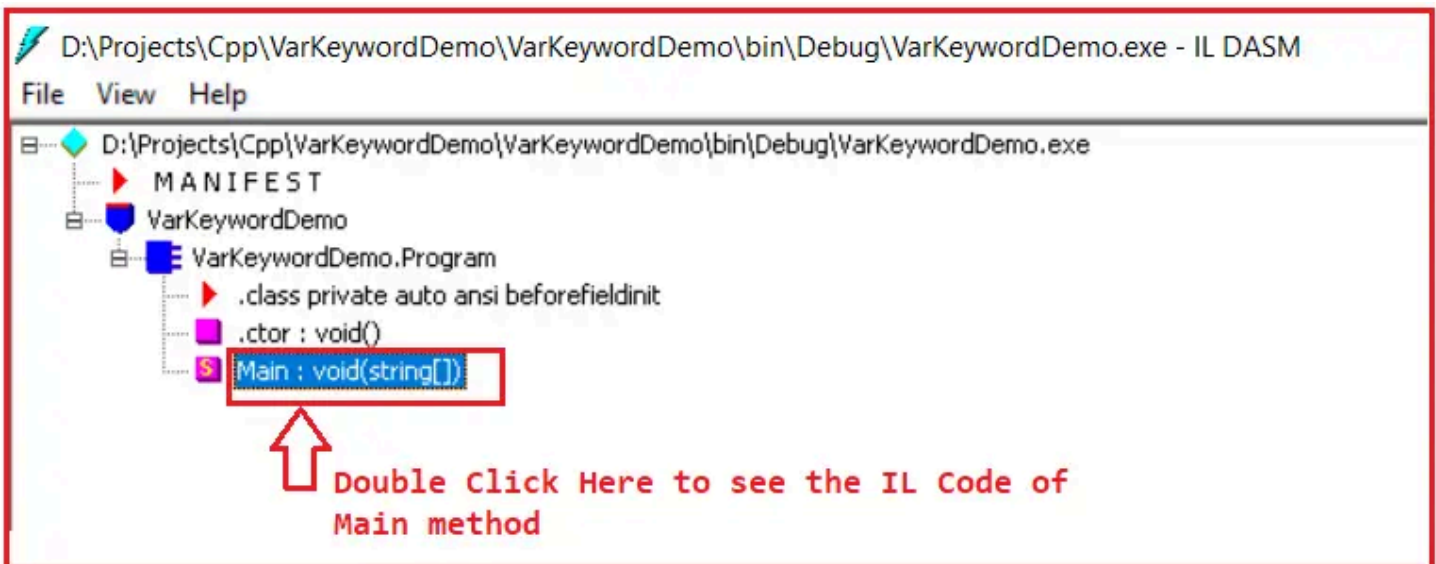
It will open the select EXE window. From this window, select the **VarKeywordDemo.exe** assembly file and then click on the Open button as shown in the below image.



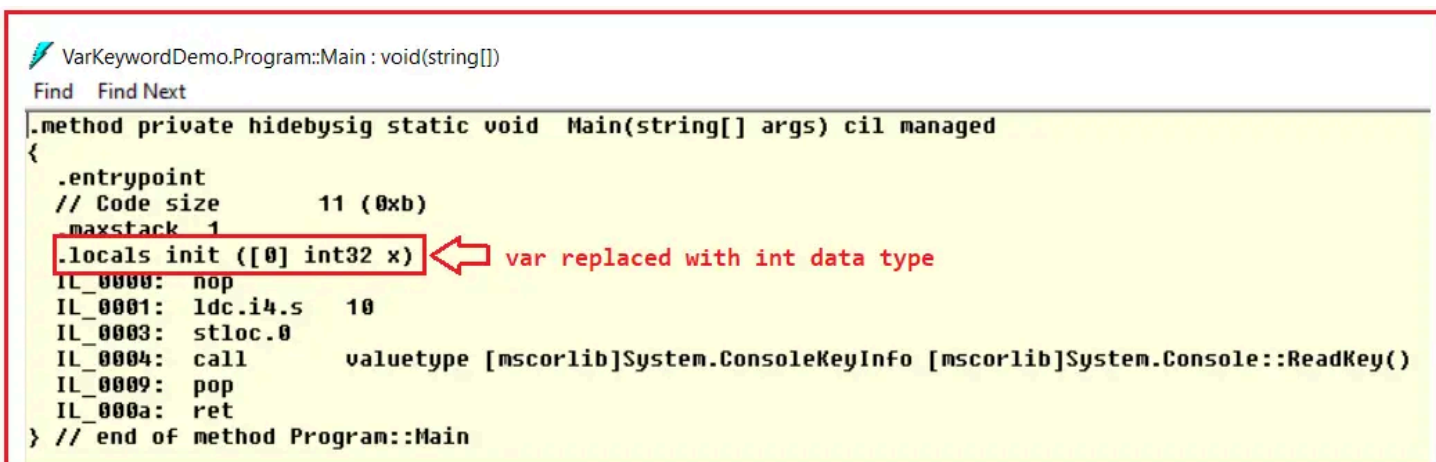
Now, you can see the **VarKeywordDemo.exe** assembly is loaded into the ILDASM window. You can expand the section by clicking on the plus button. So, after expanding, you will see the following.



Now, let us see, how does the IL (Intermediate Language) Code look like. If the var keyword defined the data type statically, then you should see “int” in the IL Code. As we have defined the declaration inside the Main method, so double-click on the main method to see the IL code of Main Method as shown in the below image.



Once you double-click on the Main method, you will see the following IL (Intermediate Language) Code of the Main method. See, it replaces the var keyword with the int data type.



So, this proves that the var keyword defined the data statically, not at runtime.

Note: The most important point that you need to keep in mind is that with the var keyword in C#, type checking and type safety are enforced at compile-time only.

What is the need for the var keyword in C#?

Now, let us understand the practical use of the var data type in C#. Declaring the variables using simple data types like int, double, bool, etc. is simpler and much clearer. Then the question that should come to your mind is when we need to use the var data type in C#. Let us understand the need and use of var data type in C# with some examples. First, create a generic big-name SomeBigClassWithSomeMoreOperations<T1, T2> class as follows:

```
namespace VarKeywordDemo
{
    public class SomeBigClassWithSomeMoreOperations<T1, T2>
    {
        public string Name { get; set; }
    }
}
```

Now, let us create an instance of the above generic class inside the Main method.

```
using System;
namespace VarKeywordDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Very Big Statement
            SomeBigClassWithSomeMoreOperations<string, string> obj = new
SomeBigClassWithSomeMoreOperations<string, string>();
            Console.ReadKey();
        }
    }
}
```

You can see the object creation statement becomes quite long and also unreadable. With the var keyword, the code becomes short and sweet and also becomes readable as shown in the below code.

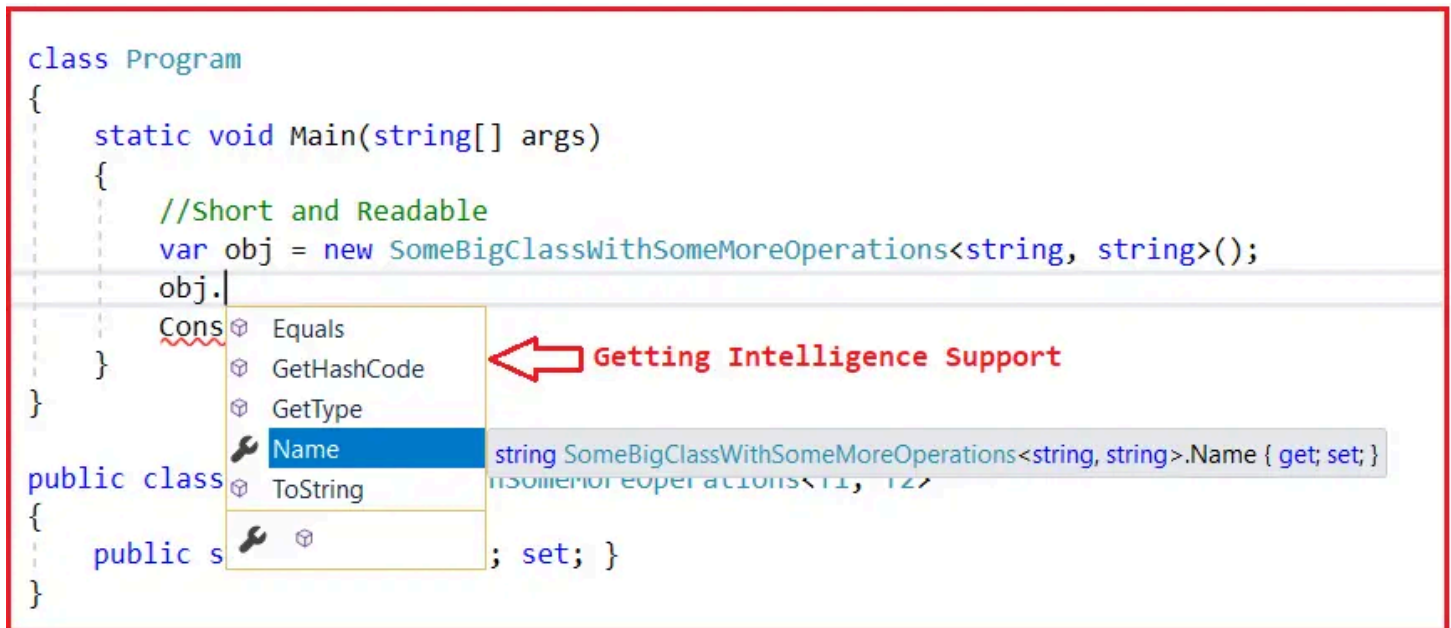
```
using System;
namespace VarKeywordDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Short and Readable
            var obj = new SomeBigClassWithSomeMoreOperations<string, string>()
```

```

        Console.ReadKey();
    }
}

```

So, this is one of the use cases of the var keyword when your class name is big. Not only does the var keyword make your code short and readable but also it provides intelligence support and compile-time error checking. As the class contains one public property i.e. Name, you can see that the intelligence shows the public property of the class as well as the members of the object class when you type obj dot (.) as shown in the below image.



Var Keyword used in LINQ and Anonymous Types in C#:

Another use case of the var keyword is that it is used with LINQ and Anonymous Types in C#. Let us understand this with an example. So, what we are going to do is, we will create a string array, and on the string array we will use LINQ queries and we will see how the var data type is useful.

Please have a look at the following code. Here, first, we created a string array with some names. And then we fired the LINQ query on the string array. So, basically, we need to write a LINQ query to fetch the names which are greater than 5 characters. Here, we have written the LINQ query which will return the name which is greater than 5 characters as well as the length of the name. As we don't know which type of data the LINQ query is going to return, so we use object type.

```

using System;
using System.Linq;
namespace VarKeywordDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Use LINQ with Anonymous Type

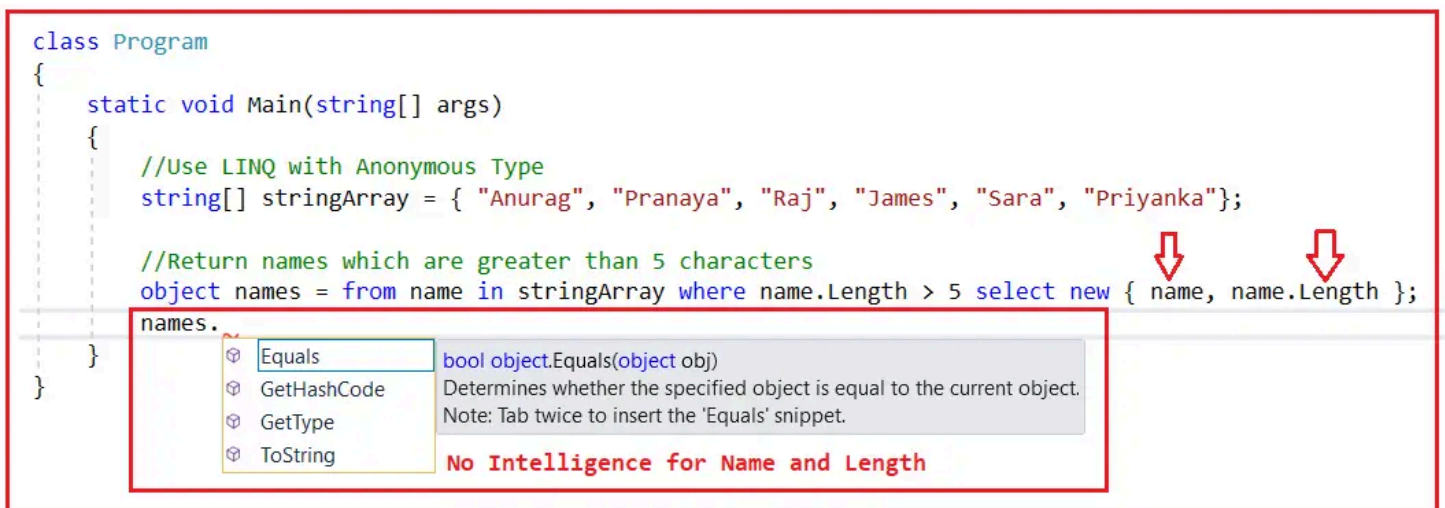
```

```

        string[] stringArray = { "Anurag", "Pranaya", "Raj", "James", "Sar
        "Priyanka" };
        //Return names that are greater than 5 characters
        //using LINQ Query Syntax
        object names = from name in stringArray
                        where name.Length > 5
                        select new
                        {
                            name,
                            name.Length
                        };
        //Return names that are greater than 5 characters
        //Method Syntax
        object names2 = stringArray.Where(name => name.Length > 5).
                                Select(name => new
                                {
                                    name,
                                    name.Length
                                });
    }
}
}

```

As the query returns the name and length of the name, you are under the assumption that when we type the name. (dot) it will give us intelligence for both name and length. But this is not the case. You will not get any intelligence except from the object class members as shown in the below image.



One of the ways to get intelligence support is to use strongly typed data types. So, what we can do is, we need to define our own class with two properties for Name and Length. And then we need to use that custom class in the LINQ query as shown in the below code.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace VarKeywordDemo
{

```

```

class Program
{
    static void Main(string[] args)
    {
        //Use LINQ with Anonymous Type
        string[] stringArray = { "Anurag", "Pranaya", "Raj", "James", "Sar
        "Priyanka" };
        //Return names that are greater than 5 characters
        //using LINQ Query Syntax
        IEnumerable<MyData> names = from name in stringArray
                                   where name.Length > 5
                                   select new MyData
                                   {
                                       Name = name,
                                       Length = name.Length
                                   };
        //Return names that are greater than 5 characters
        //Method Syntax
        IEnumerable<MyData> names2 = stringArray.Where(name => name.Length > 5)
                                                .Select(name => new MyData
                                                {
                                                    Name = name,
                                                    Length = name.Length
                                                });
        //Accessing the Data using Foreach Loop
        foreach (MyData item in names)
        {
            Console.WriteLine($"Name={item.Name} and Length = {item.Length}");
        }
        Console.ReadKey();
    }
}

public class MyData
{
    public string Name { get; set; }
    public int Length { get; set; }
}

```

Output:

```

Name=Anurag and Length = 6
Name=Pranaya and Length = 7
Name=Priyanka and Length = 8

```

With this code, you will get intelligence support as shown in the below image. Not only intelligence support, but if you mistype the names then you will also get a compile-time error.

```

class Program
{
    static void Main(string[] args)
    {
        //Use LINQ with Anonymous Type
        string[] stringArray = { "Anurag", "Pranaya", "Raj", "James", "Sara", "Priyanka"};

        //Return names which are greater than 5 characters
        IEnumerable<MyData> names = from name in stringArray where name.Length > 5
                                   select new MyData { Name =name, Length = name.Length };
        foreach (MyData item in names)
        {
            item.
        }
        Console.R
    }
}

public class MyDa
{
    public int Length { get; set; }
    public string Name { get; set; }
}

```

Getting Intelligence Support and Type Checking

Equals
GetHashCode
GetType
Length
Name
ToString

int MyData.Length { get; set; }

As you see, here we are doing a lot of hard work. We are creating a class with the required properties. Then we are using the IEnumerable collection and in the LINQ query, we are also using the custom class and properties to store the values, and then only we are getting the intelligence support. Instead of doing the above things, we can simplify the things using the var keyword and which is very simpler and easier. Let us see how we can do this by using the var keyword.

Please have a look at the following example. Here we are not using any custom class but then also we are getting intelligence support and type checking at compilation time. And this is possible because of the var keyword.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace VarKeywordDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Use LINQ with Anonymous Type
            string[] stringArray = { "Anurag", "Pranaya", "Raj", "James", "Sar
            "Priyanka" };
            //Return names that are greater than 5 characters
            //using LINQ Query Syntax
            var names = from name in stringArray
                        where name.Length > 5
                        select new

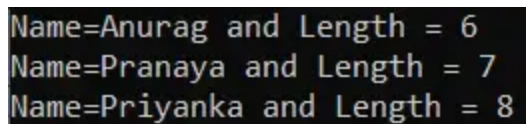
```

```

        {
            Name = name,
            Length = name.Length
        };
//Return names that are greater than 5 characters
//Method Syntax
var names2 = stringArray.Where(name => name.Length > 5).
    Select(name => new
    {
        Name =name,
        Length = name.Length
    });
//Accessing the Data using Foreach Loop
foreach (var item in names)
{
    Console.WriteLine($"Name={item.Name} and Length = {item.Length}");
}
Console.ReadKey();
}
}
}

```

Now, if you run the above code, then you will also get the same output as the previous example as shown in the below image.

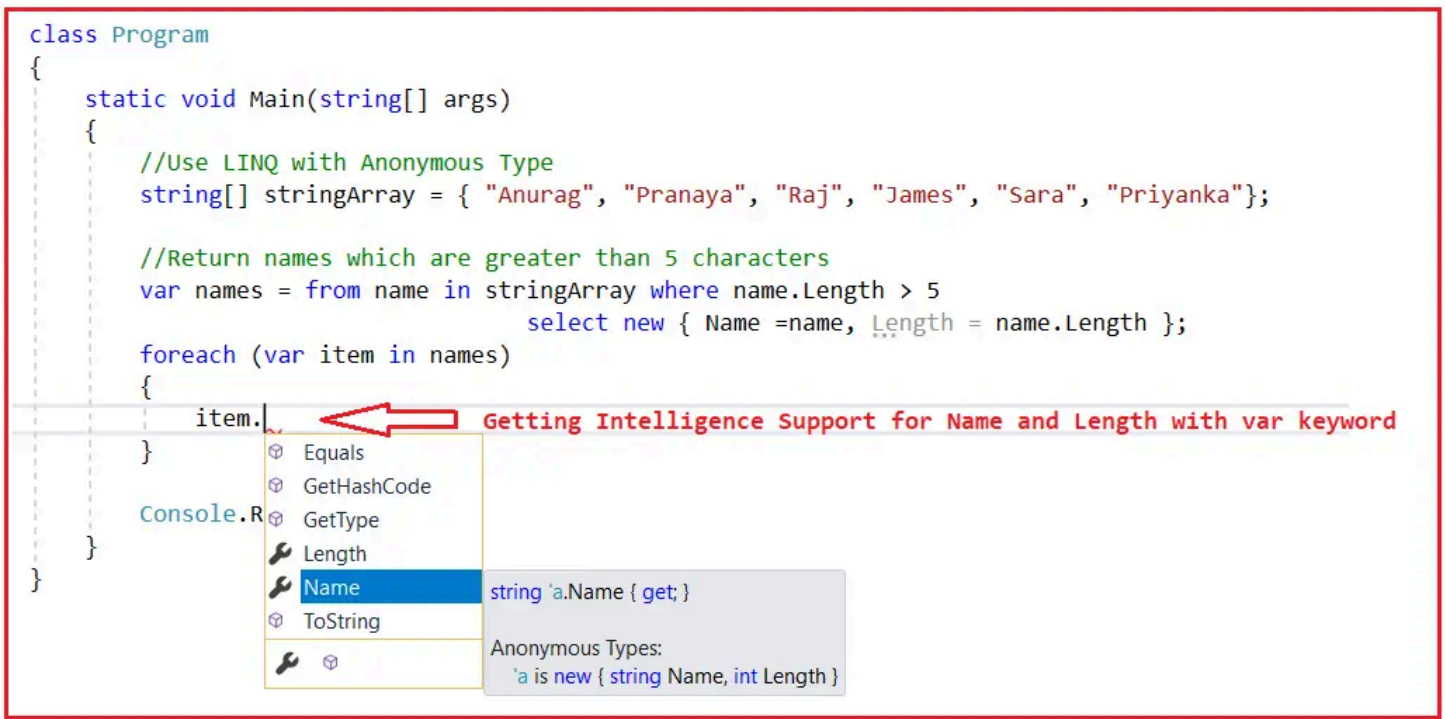


```

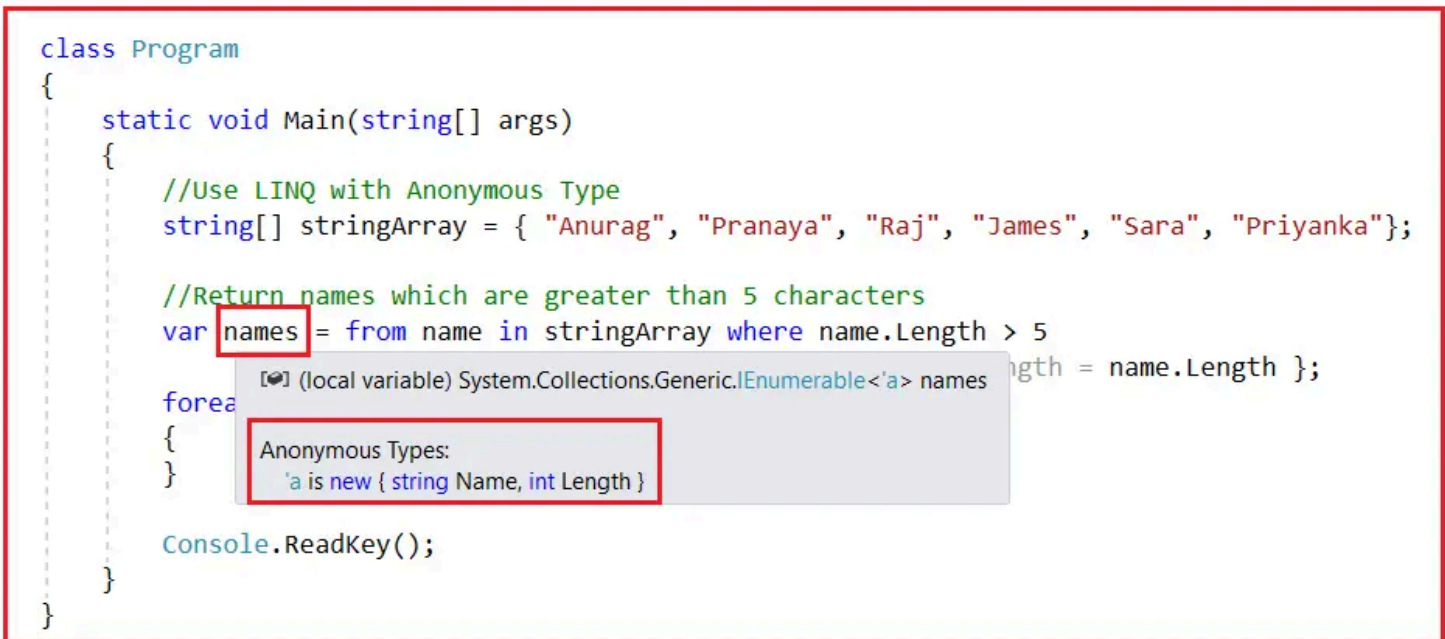
Name=Anurag and Length = 6
Name=Pranaya and Length = 7
Name=Priyanka and Length = 8

```

Now, you can see we are getting the intelligence support for the two properties Name and Length as shown in the below image.



Here, the LINQ query returns an anonymous type with `Length` and `Name` properties. If you move the mouse pointer over the `name` variable, then you will see that the type is an anonymous type as shown in the below image.



So, in situations like this where we don't know what kind of properties or columns the LINQ query is going to return i.e. anonymous type, we can use the `var` keyword. If you use the `object`, then you will have boxing and unboxing which affect the performance as well as you will not get any intelligence support. With `var`, we don't have performance issues as boxing and unboxing are not there as well as we will get Intelligence support and compile-time error checking.

When to use the `var` keyword in C#?

The var keyword can be used in the for loop, for each loop, using statements, anonymous types, LINQ, and other places. I have shown you how to use the var keyword with LINQ queries. Now, let's see examples using the var keyword with for loop, for each loop, using statements, and anonymous types in C#.

Using var keyword to declare Anonymous Type in C#:

We can use the var keyword to hold an anonymous type in C#. For a better understanding, please have a look at the following example. In the below example, the var keyword is used to hold the anonymous type.

```
using System;
namespace VarKeywordDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using var keyword to declare Anonymous Type
            //After new keyword we have not specified the type type and hence
            //it becomes an Anonymous Type
            var student = new { Id = 1001, Name = "Pranaya" };
            Console.WriteLine($"Id: {student.Id} Name: {student.Name} ");
            Console.ReadKey();
        }
    }
}
```

Output: **Id: 1001 Name: Pranaya**

Using var Keyword in Foreach Loop:

We can use the var keyword inside the for-each loop to hold the collection items. For a better understanding, please have a look at the following example. In the below example, we are creating a variable using the var type which holds the collection items. It does not matter the collection type. Whatever the collection type, it will convert the var data type to the same type. As the collection type is a string, the var type will be converted to string type during the compilation process while generating the IL Code.

```
using System;
using System.Collections.Generic;
namespace VarKeywordDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            // List of Strings
            List<string> nameList = new List<string> { "Anurag", "Pranaya", "James", "Sara", "Priyanka" };
            //Using var Keyword in Foreach Loop
            foreach (var name in nameList)
            {
                // ...
            }
        }
    }
}
```

```

        Console.WriteLine(name);
    }

    Console.ReadKey();
}
}
}

```

Using var Keyword in C# For Loop:

We can also use the var keyword in C# For loop. For a better understanding, please have a look at the following example. Here, we create the index variable using the var keyword.

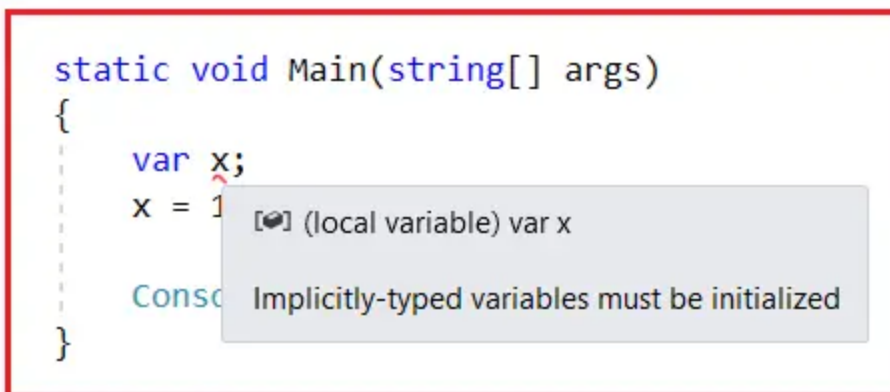
```

using System;
namespace VarKeywordDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            // Using var Keyword in For Loop
            for (var index = 1; index <= 5; index++)
            {
                Console.WriteLine(index);
            }
            Console.ReadKey();
        }
    }
}

```

Points to Remember while working with the var keyword in C#:

The variables declared using the var keyword must be declared and initialized in the same statement else we will get a compile-time error. For a better understanding, please have a look at the below image.



The variables declared using the var keyword cannot be initialized with a null value else we will get a compile-time error. For a better understanding, please have a look at the below image.

```
static void Main(string[] args)
{
    var x = null;
}
```



Cannot assign <null> to an implicitly-typed variable

Show potential fixes (Alt+Enter or Ctrl+.)

We cannot initialize the multiple implicitly-typed variables using the var keyword in the same statement. If we try, we will get a compile-time error as shown in the below code.

```
static void Main(string[] args)
{
    var x = 10, y = 20, z = 30;
}
```



Implicitly-typed variables cannot have multiple declarators

Show potential fixes (Alt+Enter or Ctrl+.)

The var keyword is not allowed to use as a field type at the class level. If we try, we will get a compile-time error as shown in the below code.

```
public class Sample
{
    var x = 20;
}
```

The contextual keyword 'var' may only appear within a local variable declaration or in script code

Advantage of using var keyword in C#

The following are the advantages of using the var keywords in C#.

1. The var keyword in C# is used to hold the result of a method whose type is not known such as an anonymous method, LINQ expressions, or generic types.
2. The most important advantage is var is type-safe, the value assigned to the var variable is known by the compiler at the compile time which prevents any issue at the runtime.
3. With the var keyword, we will get better performance as boxing and unboxing are not required.
4. It improves the code readability. It is a shorthand way of declaring a var when the class or struct names are very long.
5. The var keyword will also give Visual Studio intelligence support because the type of variable assigned is known to the compiler at the compile time.

In the next article, I am going to discuss [Var vs Dynamic in C#](#) with Examples. Here, in this article, I try to explain Var Keyword in C# with Examples. I hope you enjoy this **Var Keyword in C#** with Examples article. I would like to have your feedback. Please post your feedback, question, or comments about this article.